

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №1
«Реализация REST API на основе boilerplate»

Выполнила:

Гайдук Алина

Бр 1.2

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Реализовать работающее REST API под выбранный вариант проекта: платформу рецептов и кулинарных блогов. Опирается на уже спроектированную БД и контракт API из предыдущих заданий: перевести это в код модели, сохранение в базе, HTTP-обработчики и проверяемый сценарий использования.

Ход работы

1. Выбор бойлерплейта

Я взяла стек на Go: chi для маршрутов, GORM для объектной модели над PostgreSQL, slog для логов. Компонировку HTTP-слоя ориентировала на официальный пример chi «rest»: группы маршрутов, явное навешивание middleware. Так получился свой «мини-бойлерплейт» под RecipeHub: один модуль с точкой входа cmd/service, пакет httpserver с роутером, store с методами БД и набор хендлеров по сущностям.

2. Модели данных

- Пользователь и безопасность: User – email и username уникальны, пароль хранится только в виде хэша, есть отображаемое имя, био и аватар по желанию, временные метки created/updated. RefreshToken хранит хэш refresh-токена, срок действия и связь с пользователем с каскадным удалением, если пользователя убрали, его сессии не остаются в базе.
- Справочники: DishType, Difficulty, Tag, Ingredient – по сути id и имя (у ингредиента длиннее строка под название). MeasurementUnit – полное имя и короткое обозначение (граммы / г и т. п.), чтобы строки состава рецепта ссылались на единицу измерения.
- Рецепт и состав: Recipe привязан к автору (User), имеет заголовок, опциональное описание, обложку и видео, необязательные ссылки на тип блюда и сложность, время подготовки/готовки и порции. Вложенные сущности: RecipeStep – номер шага, текст, опциональное изображение шага; RecipeIngredient – ссылка на Ingredient, количество, опциональная единица измерения и заметка

к строке. Теги подключены связью many-to-many через таблицу `recipe_tags`. При удалении рецепта шаги и строки состава удаляются каскадом; тип блюда и сложность при удалении справочника обнуляются (`SET NULL`), чтобы не терять рецепт целиком.

- Пост блога: `Post` – автор, заголовок, текст, опциональная обложка, даты создания и обновления.
- Комментарии и социальная функция: `Comment` – автор, текст, опциональные `recipe_id` и `post_id` (комментарий относится либо к рецепту, либо к посту), `parent_comment_id` для ответов в дереве. `RecipeLike` и `PostLike` – составной первичный ключ из пары пользователь-сущность, связь с пользователем и контентом с каскадом. `Follow` – пара `follower_id` / `following_id` как ключ подписки. `SavedRecipe` – кто какой рецепт сохранил, плюс время сохранения.

3. Представления для API (DTO и маппинг)

- Общие обёртки: `TokenPair` – `access_token`, `refresh_token`, `expires_in` после регистрации/лога/refresh. `Pagination`: `total`, `limit`, `offset` для списков.
- Пользователь в ответах: `UserShort`: только публичное мини-описание (`id`, `username`, `display_name`, `avatar_url`), чтобы встраивать автора в карточки рецептов и постов без лишних полей. `UserProfile` – расширенный профиль для «себя» и карточки пользователя: `email` там, где это по смыслу эндпоинта, био – счётчики подписчиков, подписок и опубликованных рецептов, они подмешиваются при маппинге из `store`, а не дублируются колонками в модели `User`.
- Справочники в JSON – `DishType`, `Difficulty`, `Tag`, `IngredientRef`, `MeasurementUnit` как лёгкие структуры `id` + текстовые поля под фронт.
- Рецепт: `RecipeListItem` – автор как `UserShort`, тип блюда и сложность указателями, счётчики лайков и комментариев, теги, время. `RecipeFull` – полная карточка: те же поля плюс видео, шаги, ингредиенты с вложенными `IngredientRef` и `Unit`, а также флаги `is_liked` и `is_saved` для текущего зрителя.

- Пост: `PostListItem` и `PostFull`: кратко для списка, полно с `is_liked` для детальной страницы.
- Комментарий: `Comment` – автор `UserShort`, текст, `parent_comment_id`, вложенные ответы для дерева ответов.
- Вспомогательные ответы действий: `Liked`, `Following`, `Saved` – JSON после лайка/подписки/сохранения.

Маппинг `model - dto` сосредоточен в пакете `mapper`: там собираются счётчики через интерфейсы к `store` (лайки, комментарии, подписки), шаги и строки состава собираются во вложенные структуры ответа, для комментариев строится дерево ответов и ветки упорядочиваются по времени создания.

Хендлеры (контроллеры HTTP)

Каждая группа маршрутов вынесена в свой подпакет `internal/api/handler/...` с конструктором `New(deps)`, который получает общие зависимости (`store`, конфиг).

- `register`, `login`, `refresh`: разбор JSON-тела, вызов слоя БД и JWT, установка статусов и тел ответа с `TokenPair`.
- `users`: `PATCH /users/me`, `GET /users/{id}` и списки рецептов/постов/подписчиков/подписок пользователя, `POST/DELETE follow` с проверкой существования цели и того, что пользователь не подписывается сам на себя.
- `recipes`: `GET` списка с `query`-параметрами (фильтры, поиск, пагинация), `GET` по `id` с опциональным JWT для `is_liked/is_saved`, `POST/PATCH/DELETE` для автора рецепта; побочные операции вынесены в `likes` и `saved`.
- `posts`: аналогично рецептам для блога: список, деталь, CRUD для автора.
- `comments`: списки для рецепта и поста, создание с поддержкой `parent_id`, удаление своего комментария по `id`.
- `likes`, `saved`: хендлеры переключения лайка и сохранения рецепта/лайка поста.
- `refs`: только справочники – списки и создание новых записей справочников там, где по роутеру нужна авторизация.

Типичный поток внутри хендлера: прочитать контекст запроса (путь, query, тело), при необходимости достать user id из middleware авторизации, вызвать метод store, преобразовать результат через mapper в dto, отдать api.JSON или код ошибки. Роутер в `httpserver/router.go` связывает пути с этими функциями и навешивает Auth Required / Optional где нужно.

Вывод

В ходе выполнения данной лабораторной у меня получилось создать рабочее API из спецификации, созданной в домашней работе №2. Интересно было развести три слоя (model, dto и mapper) и увидеть, как от этого упрощаются изменения: можно поменять представление ответа или добавить счётчик, не переписывая всю бизнес-логику в хендлере.